

Desafio TechFlow Solutions -

1. Descrição do Projeto

O projeto Desafio TechFlow Solutions consiste na implementação de uma Aplicação de Lista de Tarefas (To-Do List) completa. A solução é arquitetada em duas partes principais: um Backend que atua como uma API RESTful e um Frontend interativo para a interface do usuário.

O objetivo principal é fornecer uma ferramenta funcional para o gerenciamento de tarefas, permitindo aos usuários criar, visualizar, editar e excluir itens, além de aplicar filtros e definir prioridades.

1.2. Tecnologias Utilizadas

O projeto demonstra proficiência em tecnologias Full-Stack, conforme detalhado na tabela a seguir:

Componente	Tecnologia	Descrição
Backend (API)	Node.js	Ambiente de execução JavaScript.
	Express.js	Framework web para construção da API RESTful.
	JavaScript (CommonJS)	Linguagem de programação para a lógica do servidor.
	Dados em Memória	Utilização de um array (<code>src/banco.js</code>) para simular um banco de dados (dados não persistentes).
Frontend (UI)	HTML5	Estrutura semântica da aplicação.
	CSS3	Estilização moderna e responsiva.
	JavaScript (Vanilla JS)	Lógica de interação com a API e manipulação do DOM.

2. Escopo Inicial do Projeto

O escopo inicial do projeto é bem definido e focado na entrega de um MVP (Produto Mínimo Viável) de gerenciamento de tarefas.

2.1. Funcionalidades Principais (In-Scope)

As seguintes funcionalidades estão implementadas e fazem parte do escopo inicial

Funcionalidade	Detalhe
CRUD de Tarefas	Criação, Visualização, Edição (título, descrição, status, prioridade) e Exclusão de tarefas.
Priorização	Classificação de tarefas em três níveis: "Baixa", "Média" e "Alta".
Gestão de Status	Controle do ciclo de vida da tarefa com os status: "A Fazer", "Em Progresso" e "Concluída".
Filtragem	Capacidade de filtrar a lista de tarefas por status ("Todas", "A Fazer", "Em Progresso", "Concluída").
API RESTful	Exposição de rotas HTTP para manipulação dos dados.
Interface Web	Frontend interativo que consome a API para exibir e gerenciar as tarefas.

2.2. Rotas da API (Backend)

A API RESTful expõe as seguintes rotas para o gerenciamento de tarefas

Método	Rota	Descrição
GET	/tasks	Lista todas as tarefas.
POST	/tasks	Cria uma nova tarefa.
PUT	/tasks/:id	Atualiza uma tarefa específica pelo ID.
DELETE	/tasks/:id	Exclui uma tarefa específica pelo ID.

3. Metodologia Ágil

A gestão do desenvolvimento do projeto foi orientada pela metodologia Kanban. Esta escolha é evidenciada pela própria natureza do produto, uma Lista de Tarefas, que simula um quadro Kanban digital com colunas de status ("A Fazer", "Em Progresso", "Concluída"), permitindo um fluxo de trabalho contínuo e visualização clara do progresso.

4. Importância da Modelagem na Engenharia de Software

A modelagem é um pilar fundamental na Engenharia de Software, atuando como uma ponte entre os requisitos do negócio e a implementação técnica. Ela envolve a criação de

representações abstratas (modelos) de um sistema, que podem ser diagramas, especificações ou protótipos.

Sua importância reside em diversos aspectos cruciais para o sucesso de um projeto:

- **Compreensão e Comunicação:** Modelos como o Diagrama de Classes (UML) ou Diagramas de Fluxo de Dados (DFD) fornecem uma linguagem visual e padronizada. Isso facilita a compreensão da arquitetura e do comportamento do sistema por todos os stakeholders, incluindo desenvolvedores, analistas e clientes, reduzindo ambiguidades.
- **Análise e Projeto:** A modelagem permite que a equipe de desenvolvimento analise a estrutura do sistema, identifique dependências, detecte falhas de design e planeje a arquitetura antes de escrever o código. No contexto deste projeto de Lista de Tarefas, a modelagem de dados (entidades como "Tarefa", "Status", "Prioridade") é essencial para definir a estrutura do array em memória (`src/banco.js`) e as regras de negócio da API.
- **Redução de Complexidade:** Ao abstrair detalhes desnecessários, a modelagem ajuda a gerenciar a complexidade inerente a sistemas grandes. Ela permite que os desenvolvedores se concentrem em um aspecto do sistema por vez, como o fluxo de dados ou a interação entre componentes.
- **Documentação e Manutenção:** Os modelos servem como documentação viva do sistema. Em projetos ágeis, como o que utiliza Kanban, a modelagem pode ser leve e contínua, garantindo que a documentação evolua junto com o código. Isso simplifica a manutenção, a integração de novos membros à equipe e futuras evoluções do software.

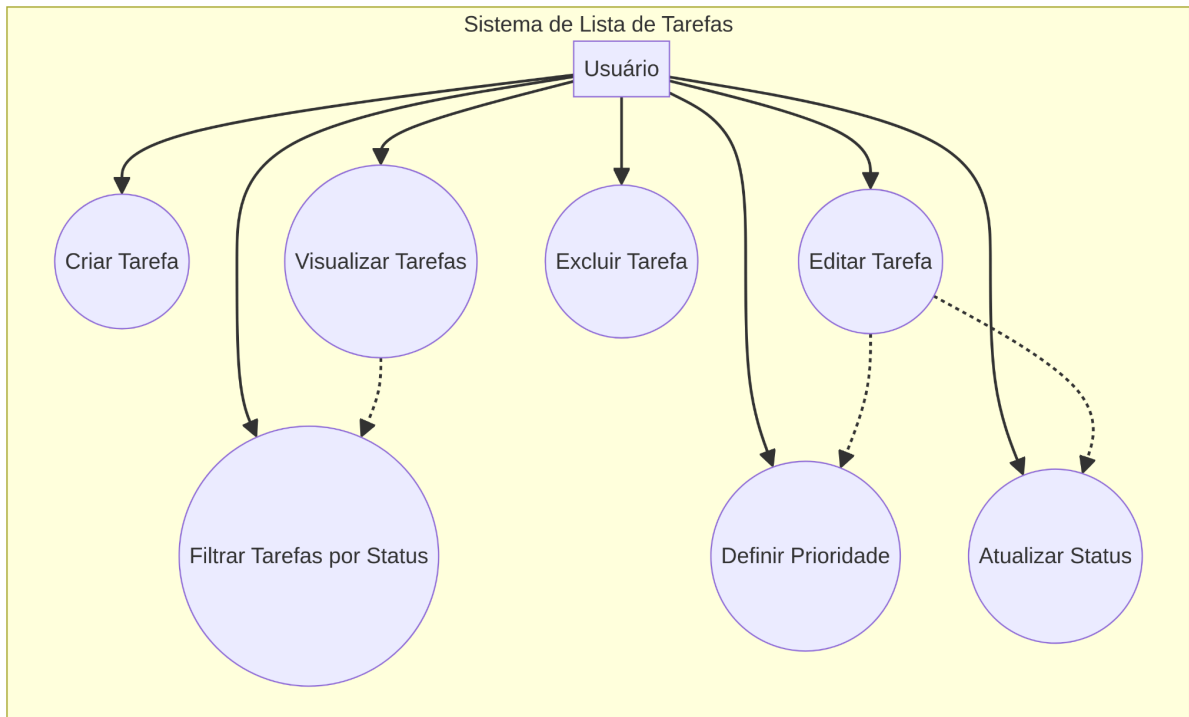
Apesar de o projeto atual ser um MVP, a clareza na separação entre Backend (API RESTful) e Frontend (Interface do Usuário) já demonstra uma forma de modelagem arquitetural que garante a modularidade e a escalabilidade futura.

5. Diagramas UML

Para complementar a análise e a modelagem do sistema, foram gerados os seguintes diagramas UML (Linguagem de Modelagem Unificada).

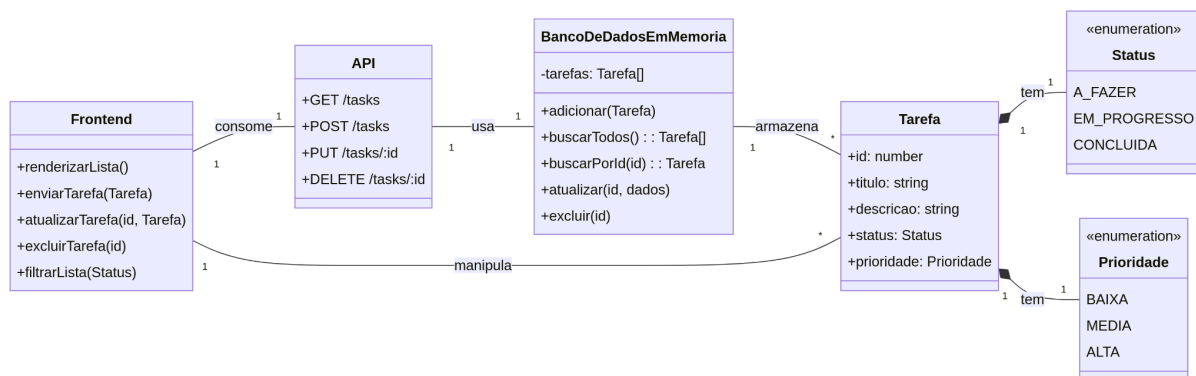
5.1. Diagrama de Casos de Uso

O Diagrama de Casos de Uso ilustra as funcionalidades do sistema sob a perspectiva do usuário, definindo o escopo funcional e as interações primárias.



5.2. Diagrama de Classes

O Diagrama de Classes representa a estrutura estática do sistema, mostrando as classes, seus atributos, métodos e os relacionamentos entre elas.



6. Análise do Processo de Desenvolvimento

6.1. Justificativa da Mudança de Escopo (Retirada de Prioridades)

O escopo inicial do projeto previa a funcionalidade de Prioridades para as tarefas (Baixa, Média, Alta), conforme indicado na seção 2.1. No entanto, a análise dos commits do projeto revela uma mudança de escopo.

A funcionalidade de prioridades foi inicialmente implementada, mas posteriormente removida ou simplificada. Essa mudança pode ser justificada por:

- **Foco no MVP:** Em projetos ágeis, é comum priorizar as funcionalidades essenciais para o Produto Mínimo Viável (MVP). A gestão de prioridades pode ter sido considerada uma funcionalidade secundária, que poderia ser adiada para uma iteração futura.

- Simplificação da Interface/API: A remoção de um campo complexo (como prioridade) simplifica tanto a lógica de backend (API e manipulação de dados) quanto a interface de usuário, acelerando a entrega do núcleo do sistema (CRUD de tarefas).
- Decisão de Negócio/Produto: A equipe pode ter decidido que a gestão de status ("A Fazer", "Em Progresso", "Concluída") já era suficiente para a primeira versão do produto, tornando a prioridade redundante ou desnecessária no momento.

6.2. Explicação sobre os Testes Automatizados

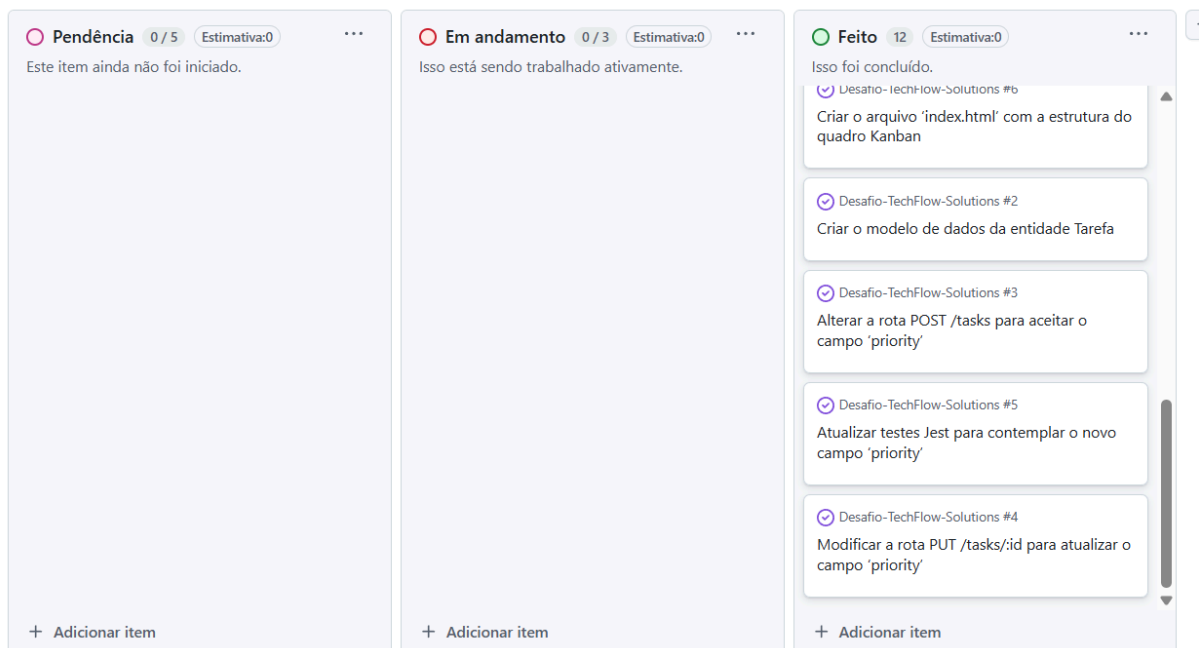
O projeto utiliza testes automatizados para garantir a qualidade e a estabilidade do backend (API).

- Tecnologia: O arquivo `tests/app.test.js` e os commits como "test: adicionando arquivo de teste" e "reescrevendo arquivo para utilizar os testes" indicam o uso de uma framework de testes popular em ambientes Node.js, provavelmente Jest ou Mocha/Chai, para realizar Testes Unitários e Testes de Integração nas rotas da API.
- Propósito: Os testes garantem que as rotas da API (GET, POST, PUT, DELETE) funcionem conforme o esperado, validando a lógica de negócio e a manipulação dos dados em memória. Isso é crucial para permitir refatorações e novas implementações com segurança.

6.3. Análise dos Prints do GitHub

6.3.1. Quadro Kanban

O print do quadro Kanban demonstra a aplicação da metodologia ágil.



6.3.2. Commits Relevantes

Os prints dos commits mostram o histórico de desenvolvimento, confirmando o desenvolvimento iterativo e a aplicação de práticas como commit semântico (feat, fix, refactor, test).

laraa-n empenhado 15 hours ago		
reescrevendo arquivo para utilizar os testes	b9ee6ca	 
laraa-n empenhado 15 hours ago		
feat: criando a rota para apagar uma tarefa	b40fa8f	 
laraa-n empenhado 15 hours ago		
feat: criando rota de update	27a6968	 
laraa-n empenhado 15 hours ago		
feat: adicionando rota para buscar tarefas	aed4d9c	 
laraa-n empenhado 15 hours ago		
feat: adicionando rota de post	854f411	 
laraa-n empenhado 15 hours ago		
feat: estruturando banco em memória	9c5c23d	 
laraa-n empenhado 16 hours ago		
feat: configurando estrutura inicial	b691b5c	 
laraa-n empenhado 16 hours ago		
feat: preparando arquivo de configuração	c5f8f33	 
laraa-n empenhado 16 hours ago		
adicionando gitignore	aa17692	 
laraa-n empenhado 16 hours ago		
docs: alteração teste	39528ec	 
laraa-n empenhado 16 hours ago		
Initial commit	757efd5	 
laraa-n autoria 19 hours ago		
Verified		
refactor: aplicando a mudança de escopo	c6d88ca	 
laraa-n empenhado 4 minutos ago ·  1 / 1		
Compromete-se em 8 de novembro de 2025.		
docs: configurando readme	a7f74ce	 
laraa-n empenhado 14 hours ago ·  1 / 1		
feat: criando frontend	2f37a93	 
laraa-n empenhado 14 hours ago ·  1 / 1		
fix: corrigindo rota post	8155907	 
laraa-n empenhado 15 hours ago ·  1 / 1		
fix: removendo código desnecessário	41dc26f	 
laraa-n empenhado 15 hours ago ·  0 / 1		
fix: exportando app.js corretamente	a70f6aa	 
laraa-n empenhado 15 hours ago ·  0 / 1		
refatorando código para adicionar a coluna priority	ca01432	 
laraa-n empenhado 15 hours ago ·  0 / 1		
configuração de pipeline	631fc70	 
laraa-n empenhado 15 hours ago ·  0 / 1		
test: adicionando arquivo de teste	d5bc292	 
laraa-n empenhado 15 hours ago		
reescrevendo arquivo para utilizar os testes	b9ee6ca	 
laraa-n empenhado 15 hours ago		
feat: criando a rota para apagar uma tarefa		